

PRACTICAL EXPERIMENT – 2

Name: K ANSHU

Roll No: 2520030440

Section: 8

Skill: Interactive Shell Input Handling, Terminal Control, Keyboard Input Processing

Title

Implementation of Interactive Shell Main Loop and Keyboard Input Handling.

Aim

To create an interactive shell main loop that displays a prompt, reads user input, handles Enter and exit conditions, and captures keyboard input including backspace and multi-character commands.

Theory

An interactive shell repeatedly displays a command prompt, accepts input from the user, and processes the entered command. Keyboard input can be handled by reading individual characters and maintaining an input buffer. Backspace can be handled by removing the last character from the buffer, while the Enter key completes the current input. An exit command can be used to terminate the shell loop.

Algorithm

1. Start the program.
2. Initialize the input buffer.
3. Configure the terminal to read keyboard input character by character.
4. Display the shell prompt.
5. Read characters from the keyboard.
6. Store normal characters in the input buffer.
7. Handle the Backspace key by removing the last character.
8. Handle the Enter key and terminate the current input.
9. Check whether the entered command is exit.
10. If the command is exit, terminate the shell.
11. Otherwise, display the entered command and repeat the loop.
12. Restore the original terminal settings and stop.

Code

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define BUFFER_SIZE 100

int main()
{
    char buffer[BUFFER_SIZE];
    int length;
    char ch;
    struct termios oldt, newt;
```

```

printf("=== OSSP Experiment 2 ===\n");
printf("Simple Shell Input Loop\n");
printf("Type a command and press Enter.\n");
printf("Type exit to quit.\n\n");

tcgetattr(STDIN_FILENO, &oldt);
newt = oldt;
newt.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSANOW, &newt);

while (1)
{
    length = 0;
    printf("ossp> ");
    fflush(stdout);

    while (1)
    {
        ch = getchar();

        if (ch == '\n')
        {
            buffer[length] = '\0';
            printf("\n");
            break;
        }
        else if (ch == 127 || ch == 8)
        {
            if (length > 0)
            {
                length--;
                printf("\b \b");
                fflush(stdout);
            }
        }
        else if (length < BUFFER_SIZE - 1)
        {
            buffer[length++] = ch;
            putchar(ch);
            fflush(stdout);
        }
    }

    if (strcmp(buffer, "exit") == 0)
    {
        printf("Exiting OSSP Shell...\n");
        break;
    }

    printf("Command entered: %s\n", buffer);
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

```

Execution Screenshots

1. Main C Code Creation

```

admin@heeraaj@DESKTOP-J39GHP7:~$
admin@heeraaj@DESKTOP-J39GHP7:~$ cat > main.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define BUFFER_SIZE 100

int main()
{
    char buffer[BUFFER_SIZE];
    int length;
    char ch;
    struct termios oldt, newt;

    printf("=== O SSP Experiment 2 ===\n");
    printf("Simple Shell Input Loop\n");
    printf("Type a command and press Enter.\n");
    printf("Type exit to quit.\n\n");

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        length = 0;
        printf("oss> ");
        fflush(stdout);

        while (1)
        {
            ch = getchar();

            if (ch == '\n')
            {
                buffer[length] = '\0';
                printf("\n");
                break;
            }
            else if (ch == 127 || ch == 8)
            {
                if (length > 0)
                {
                    length--;
                    printf("\b \b");
                    fflush(stdout);
                }
            }
            else if (length < BUFFER_SIZE - 1)
            {
                buffer[length++] = ch;
                putchar(ch);
                fflush(stdout);
            }
        }

        if (strcmp(buffer, "exit") == 0)
        {
            printf("Exiting O SSP Shell...\n");
            break;
        }

        printf("Command entered: %s\n", buffer);
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return 0;
}

```

2. Compilation and Program Execution

```
}  
EOF  
gcc main.c -o ossp  
./ossp  
=== OSSP Experiment 2 ===  
Simple Shell Input Loop  
Type a command and press Enter.  
Type exit to quit.  
  
ossp> ./ossp  
Command entered: ./ossp  
ossp> hello world  
Command entered: hello world  
ossp> d  
Command entered: d  
ossp> exit  
Exiting OSSP Shell...
```

Result

The interactive shell loop was implemented and executed successfully. The program displayed the shell prompt, accepted multi-character input, and terminated successfully when the exit command was entered.